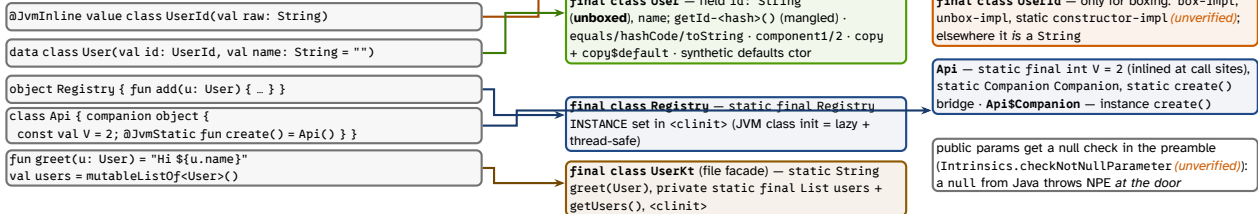


**In one line:** The JVM knows nothing about Kotlin: no top-level functions, no properties, no companions, no default arguments, no value classes, no suspend. **kotlinc** lowers each feature to plain classes, static methods, synthetic bridges and runtime helper calls (`kotlin.jvm.internal.Intrinsics`). Knowing the lowering explains Java-interop annotations, boxing costs, `$default` frames in stack traces, and why a read-only `List` can still change.

User.kt → the classes javap shows



See it yourself: IDE Tools → Kotlin → Show Kotlin Bytecode → Decompile, or `javap -c -p`.  
@file:JvmName("Users") renames the facade; internal members get name\$module names.

## Declarations → JVM

- **Properties** = private field + `getX()/setX()` (isX stays isX). `@JvmField` exposes the field; `const val` (primitive/String, top-level/object/companion) is a compile-time constant **copied into callers** — change it, recompile them.
- **Companion** = nested class `Outer$Companion` + static field; its members are *instance* methods. `@JvmStatic` adds a real static in `Outer`, so Java writes `Api.create()`.
- **Default args**: one body + synthetic `f$default(args..., int mask, Object)`; each bit of mask says “use the default”. Java sees only the full signature — add `@JvmOverloads` for telescoping overloads.
- **Lambdas**: `invokedynamic` + `LambdaMetafactory` by default since **2.0** (smaller APKs; not serializable; `-XLambdas=class` reverts). Captured vars become `Ref.IntRef/ObjectRef`.
- **lateinit var**: non-null field; the getter null-checks and throws `UninitializedPropertyAccessException`; `::x.isInitialized`; no primitives. A nullable var instead makes every read `?./!!`.
- No checked exceptions in Kotlin: without `@Throws(IOException::class)` Java cannot catch (`IOException e`) around the call — “never thrown in try”.

## Example — Java-facing API

```

class Client @JvmOverloads constructor(
    val url: String, val retries: Int = 3, val gzip: Boolean = true) {
    companion object {
        const val VERSION = 2 // Java: Client.VERSION (inlined)
        @JvmField val TAG = "client" // Java: Client.TAG (static field)
        @JvmStatic fun local() = Client("https://example.com")
        fun plain() = 1 // Java: Client.Companion.plain()
    }
    @Throws(IOException::class) fun fetch(): ByteArray = TODO()
}
val a: Int? = 1000; val b: Int? = 1000
a == b // true -> equals()
a === b // false -> two Integer boxes (cache is -128..127)
  
```

## Control flow & collections

- when (s) on **sealed/is**: an instance of chain; on JVM 21+ a single `invokedynamic SwitchBootstraps.typeSwitch` (experimental 2.2.20, **default since 2.4.20**). On an **enum**: synthetic `$WhenMappings` ordinal table + `tableswitch (unverified)`.
- **Read-only ≠ immutable**: `List` is a compile-time view of `java.util.List`; `listOf(a, b)` is `Arrays.asList`-backed (*unverified*), `mutableListOf` an `ArrayList` — a cast or Java code can mutate it. `setOf` iterates in insertion order.
- **Iterable** ops are eager — every step builds a new list. **Sequence** is lazy, *per element* (`map 1 → filter 1 → ...`), short-circuits (*first*); pays off for long chains/big data, not 10 items or `sorted()`.

## Boxing

- `Int → int`; `Int?, List<Int>`, generic `T → Integer`.  
`IntArray = int[], Array<Int> = Integer[]`. `===` on boxes compares objects.

## Compiler & targets

- **K2** (stable, default since **2.0**, May 2024): new frontend **FIR** feeds a shared **IR** backend → JVM bytecode, JS, Wasm, or LLVM (Native).
- **Kotlin/Native**: LLVM machine code, a **tracing GC** (new memory manager, default since 1.7.20, no freeze); ARC only at the ObjC/Swift edge.

## Traps

- ! “`List` is immutable” — only a read-only *interface*.
- ! `foo$default` in a trace = the default-args bridge.
- ! Changed a library `const val`: callers keep the *old* inlined value.

**Remember:** `FileKt · INSTANCE · Outer$Companion · $default + mask · indy lambdas · List = java.util.List`.